

WEEK 7: MINIMUM CUT

Advanced Algorithms

ROADMAP

1. Randomised Algorithms
2. Basic Probability
3. Minimum Cut and Edge Contraction
4. Karger's Algorithm
5. The Karger–Stein Algorithm

This week the algorithm gets to *flip coins* — and we will prove how likely it is to be right.

RANDOMISED ALGORITHMS

A MOTIVATING EXAMPLE

Say that we have a vector of size 1,000,000 where each entry is zero or one.

We are **promised** that at least 500,000 entries are one.

The goal is to output the index of an entry that is one.

Scanning from the left works... but in the worst case reads 500,001 entries.

USING RANDOMNESS

A randomised algorithm is allowed to flip coins and use the outcomes to decide on its next action.

Different runs of a randomised algorithm on an input can produce *different* outputs.

We don't require a randomised algorithm to always output the right answer, but instead that it outputs the right answer at least $2/3$ of the time.

The exact fraction doesn't matter too much — we will see how to boost it.

A RANDOMISED SOLUTION

How could we use coin flips to find a one in the input?

Choose a random entry: since at least half the entries are one, the probability it is a one is at least $1/2$.

What is the chance we don't see a one after looking at 2 random entries? This is like the chance of not seeing heads after flipping a coin twice.

HH HT TH TT

Of the four outcomes only one has no head: the chance of failure is $1/4$.

DRIVING DOWN THE ERROR

Similarly, the chance of not seeing a head after 3 coin flips is $1/8$ — the only "bad" outcome is TTT.

By randomly looking at t entries we reduce the failure probability to $1/2^t$.

CONTRAST Any deterministic algorithm must read $n/2 + 1$ entries to always succeed. The randomised algorithm reads 30 entries and fails with probability 2^{-30} .

REPEAT UNTIL SUCCESS

Notice that when we query an entry we can see whether it is a one: success is recognisable.

So there is no need to fix a budget in advance — keep querying random entries until we find a one.

This algorithm is **never** wrong. What varies from run to run is how long it takes.

Each query finds a one with probability at least $1/2$. We will see that the *expected* number of queries to see a one is 2.

LAS VEGAS ALGORITHMS

DEFINITION A randomised algorithm is *Las Vegas* if it always outputs the correct answer, and its running time is a random variable that we bound in expectation.

No matter what the random coins, the answer is right; depending on the random coins it may take longer or shorter to get there.

There is no worst-case bound here: an unlucky run can query for a long time. It halts with probability 1, and what we control is the *expected* number of queries.

A MORE GENERAL SETTING

Now say the input is an array of n integers.

We want to find the *minimum value* of the array, and we are guaranteed the minimum value appears at least d times.

Unlike the previous problem, we don't know when we have succeeded — we can't recognise the minimum when we see it.

So repeating until success is not an option: we must fix the number of queries in advance and live with the chance of a wrong answer.

A GENERIC RANDOMISED ALGORITHM

Randomly query T entries and return the smallest value amongst them.

It only fails if the minimum value is not amongst the entries queried.

How large should T be to ensure this happens only with small probability?

Monte Carlo Algorithms

DEFINITION A randomised algorithm is *Monte Carlo* if, on every input, it halts within a worst-case time bound and outputs the correct answer with probability at least $2/3$.

No matter what the random coins, the algorithm will halt within a specified time; depending on the random coins it may get the right answer or not.

Karger's algorithm will be exactly of this kind.

TWO KINDS OF GUARANTEE

	ANSWER	RUNNING TIME
Las Vegas	always correct	random
Monte Carlo	may be wrong	worst-case bound

BASIC PROBABILITY

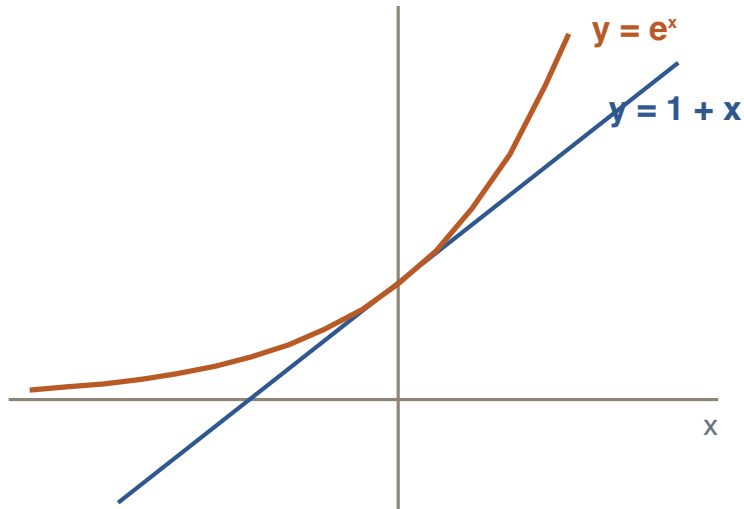
COIN FLIPS

Let $p = d/n$. When we randomly choose an entry, the probability it is the minimum value is at least p .

If we have a coin that lands heads with probability p , how many times do we have to flip it to see a heads with probability at least $9/10$?

FACT By independence of the flips, if we toss the coin t times the probability we do *not* see a head is $(1 - p)^t$.

A USEFUL INEQUALITY



FACT $1 + x \leq e^x$ for any real x .

Here $e = 2.718 \dots$ is Euler's constant,

$$e = \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n} \right)^n.$$

The line $1 + x$ is tangent to e^x at $x = 0$ and the curve is convex — that is the whole proof.

APPLYING THE INEQUALITY

Taking $x = -p$ gives $1 - p \leq e^{-p}$, and so

$$(1 - p)^t \leq e^{-pt}.$$

If we take $t = 1/p$ flips, the failure probability is at most

$$\frac{1}{e} \leq 0.368.$$

Taking $t = \lceil \ln(10) \rceil / p \leq 3/p$ flips pushes the failure probability below $1/10$.

BACK TO FINDING THE MINIMUM

An array of n integers whose minimum appears at least d times; output the minimum.

Each random query lands on the minimum with probability $p \geq d/n$.

CONCLUSION Query $3n/d$ random entries and return the smallest. This is wrong with probability at most $1/10$.

This is a Monte Carlo algorithm: a fixed budget of queries, and a small chance of a wrong answer.

AMPLIFICATION

The argument is general, as long as a correct answer can be picked out of a batch of runs whenever one is present — here, the smallest value returned.

FACT If each run succeeds with probability at least p , then $t \geq 3/p$ independent runs contain a successful one with probability at least $9/10$.

Proof: all t runs fail with probability at most $(1 - p)^t \leq e^{-pt} \leq e^{-3} < 1/10$.

We will use this twice more today, for Karger's algorithm and its refinement.

BACK TO LAS VEGAS

So far we have fixed the number of queries t in advance and bounded the chance of failure by $(1 - p)^t$.

Now let's return to the Las Vegas algorithm for finding a one: query random entries until we see a one.

This algorithm never fails. What is random is the number of queries it makes — and we want to bound this on average.

In coin language: a coin lands heads with probability p . How many flips until the first heads, on average? Making "on average" precise is the job of [expectation](#).

EXPECTATION

Let us look at the **expected** number of times we have to flip the coin before we see heads. Let X be a variable representing the number of flips until the first heads.

DEFINITION The expectation of X is a weighted average:

$$\mathbb{E}[X] = 1 \cdot \Pr[X = 1] + 2 \cdot \Pr[X = 2] + \dots$$

THE GEOMETRIC DISTRIBUTION

In our case of a coin that is heads with probability p we have

$$\Pr[X = i] = (1 - p)^{i-1}p.$$

So

$$\mathbb{E}[X] = 1 \cdot p + 2 \cdot (1 - p)p + 3 \cdot (1 - p)^2p + \dots$$

This is tricky, as it is *not* a geometric series...

GEOMETRIC SERIES

Let's start with an easier case, the geometric series

$$p + (1 - p)p + (1 - p)^2p + (1 - p)^3p + \dots$$

OBSERVATION The i th term is exactly $\Pr[X = i]$. As the sum of probabilities of disjoint events that cover all outcomes, this series is 1.

BREAK DOWN

We break $\mathbb{E}[X]$ down as a sum of geometric series — the i th row is a geometric series starting at $(1 - p)^{i-1}p$, which sums to $(1 - p)^{i-1}$:

$$\begin{aligned}\mathbb{E}[X] &= 1 \cdot p + 2 \cdot (1 - p)p + 3 \cdot (1 - p)^2p + \dots \\ &= 1 \cdot p + 1 \cdot (1 - p)p + 1 \cdot (1 - p)^2p + \dots &= 1 \\ &\quad + 1 \cdot (1 - p)p + 1 \cdot (1 - p)^2p + \dots &= (1 - p) \\ &\quad \quad + 1 \cdot (1 - p)^2p + \dots &= (1 - p)^2\end{aligned}$$

FINAL EXPECTATION

Summing the rows,

$$\mathbb{E}[X] = 1 + (1 - p) + (1 - p)^2 + (1 - p)^3 + \dots$$

Multiplying by p turns this into the series that sums to 1, so

$$p \cdot \mathbb{E}[X] = 1.$$

FACT The expected number of coin flips before we see heads is $\mathbb{E}[X] = 1/p$.

BACK TO THE FIRST EXAMPLE

A vector of 1,000,000 entries, at least half of them one; output the index of a one.

Each random query lands on a one with probability $p \geq 1/2$.

CONCLUSION The Las Vegas algorithm — query until we see a one — is always correct and needs $\mathbb{E}[X] = 1/p \leq 2$ queries on average.

Compare: any deterministic algorithm must read 500,001 entries to be sure of finding a one.

PUTTING IT TOGETHER

Two things to carry forward:

- independent repetitions drive the failure probability down as $(1 - p)^t \leq e^{-pt}$ — this one returns for Karger's algorithm,
- the expected number of trials until the first success is $1/p$.

MINIMUM CUT AND EDGE CONTRACTION

CUTS

Say that we are given an undirected, unweighted graph G that is connected.

DEFINITION For every subset of vertices S there is an associated cut: the set of edges with *exactly one* endpoint in S . We denote this set of edges by $\text{cut}(S)$.

Note that S and the complement of S define the same cut.

MINIMUM CUT

The cut defined by the empty set is not interesting and we ignore it.

When the graph has n vertices there are $2^{n-1} - 1$ interesting cuts: each of the 2^n subsets defines a cut, but S and its complement coincide, and we drop $S = \emptyset$.

MINIMUM CUT

PROBLEM The minimum cut problem is to determine the smallest size of one of these cuts:

$$\text{mincut}(G) = \min_{\substack{S \\ 0 < |S| \leq n/2}} |\text{cut}(S)|$$

ORIENTATION

The **degree** of a vertex is the number of edges incident to it.

The cut of a single vertex $\{v\}$ is called a **star cut**; its size is the degree of v . So the size of a minimum cut is at most the minimum degree of a vertex.

When there are n vertices, the degree of a vertex is at most $n - 1$ — so $\text{mincut}(G) \leq n - 1$.

BRUTE-FORCE ALGORITHM

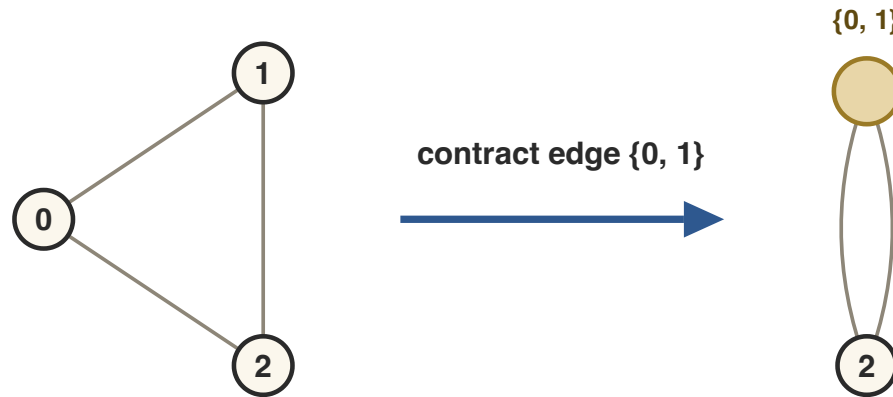
The brute-force algorithm would iterate over every cut and compute its size — time exponential in the number of vertices.

In fact, when the graph has m edges and n vertices there is a randomised algorithm that computes the minimum cut in time $O(m \log^2 n)$ [Gawrychowski, Mozes, Weimann, 2019].

TODAY A simple randomised algorithm due to **Karger** that works in time $O(n^2 m \log n)$.

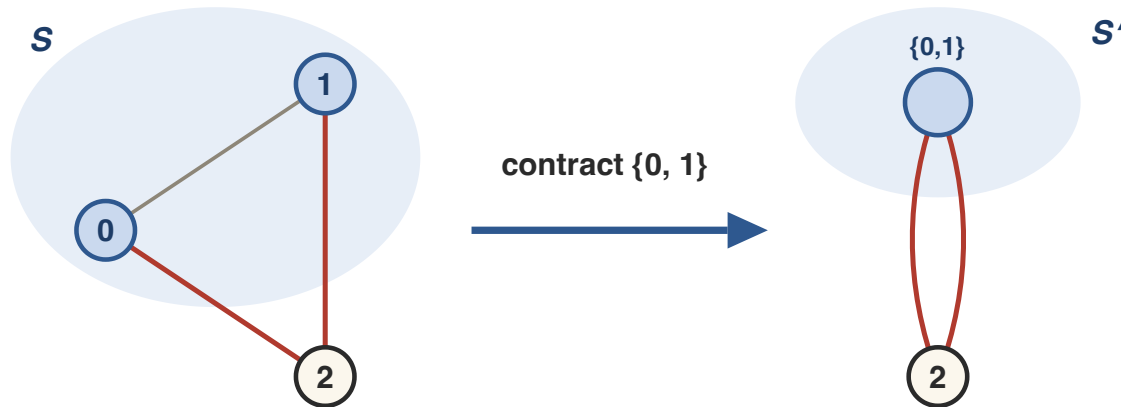
EDGE CONTRACTION

The key primitive in Karger's algorithm is **edge contraction**: delete the edge, merge its endpoints into one new "super-vertex", and keep all outgoing edges with multiplicity.



The result can be a **multigraph**, i.e. have more than one edge between two vertices. We label the super-vertex with the union of the labels of the original endpoints.

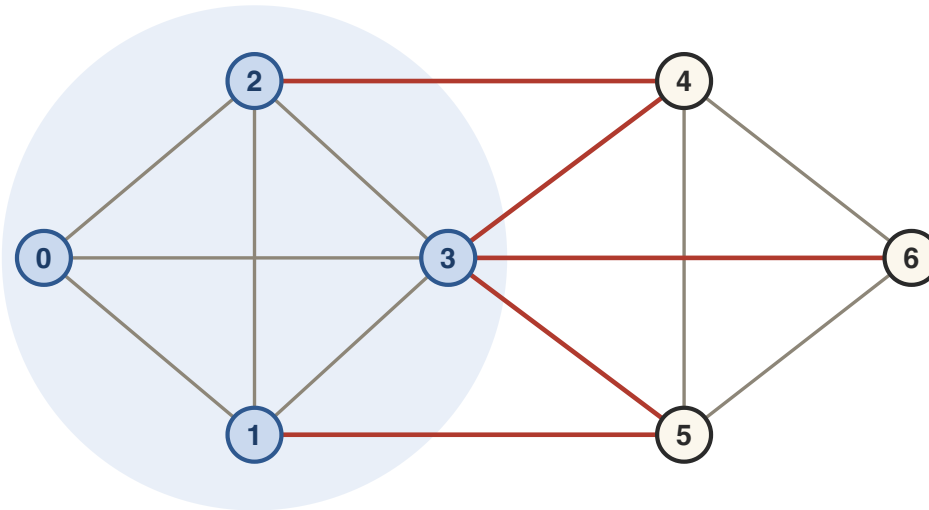
CUTS AND EDGE CONTRACTION



Any subset S containing both endpoints of the contracted edge maps to a subset S' of the new graph with $|\text{cut}(S)| = |\text{cut}(S')|$ — and every S' in the contracted graph arises this way.

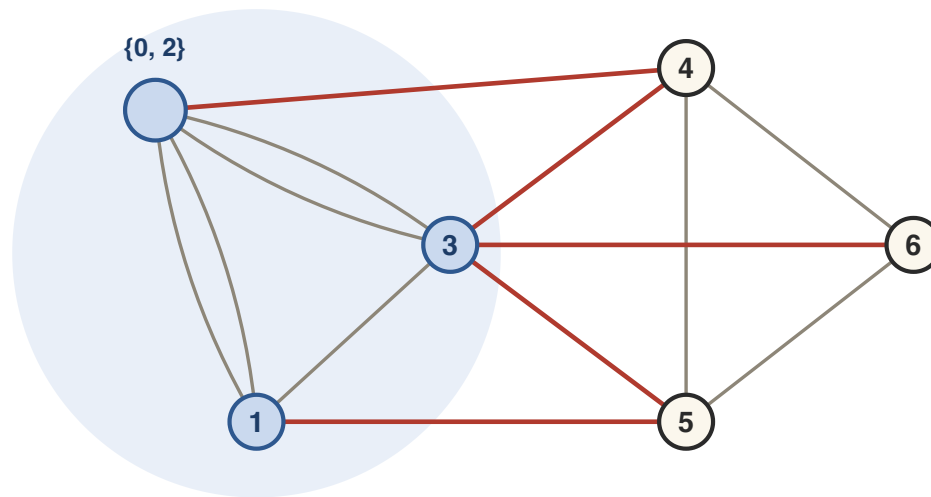
KEY FACT Edge contraction cannot decrease the size of a minimum cut.

SEQUENCE OF CONTRACTIONS



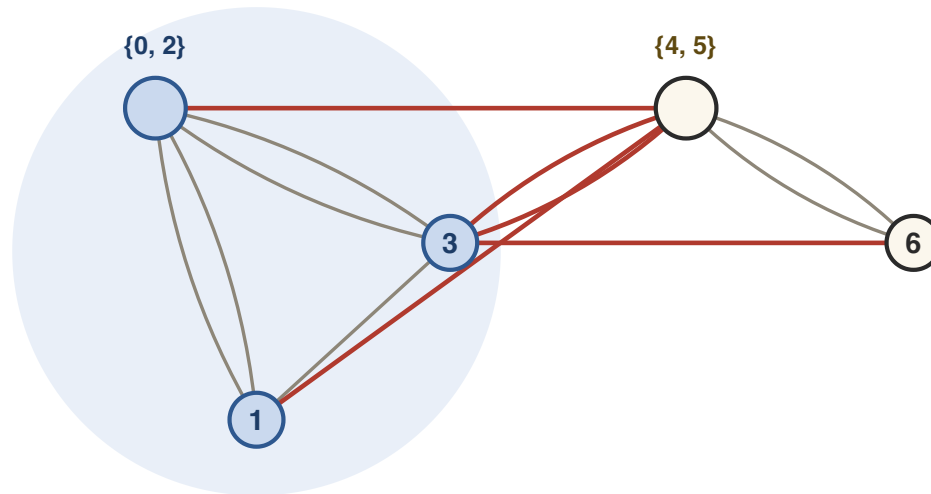
The cut defined by $S = \{0, 1, 2, 3\}$ has size 5 (the red edges). Watch this cut as we do a sequence of edge contractions.

CONTRACT THE EDGE $\{0, 2\}$



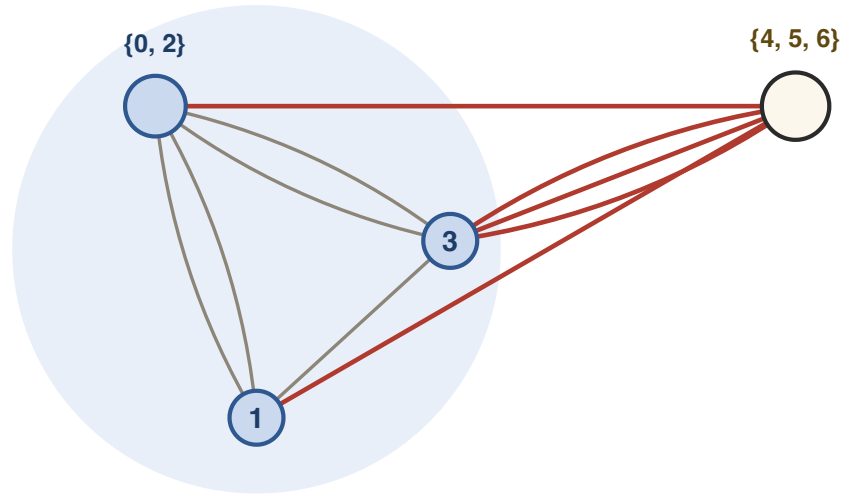
Parallel edges appear inside S . The cut defined by $\{\{0, 2\}, 1, 3\}$ still has size 5.

CONTRACT THE EDGE $\{4, 5\}$



Contractions on the *other* side of the cut are harmless too: the cut defined by $\{\{0, 2\}, 1, 3\}$ still has size 5.

CONTRACT THE EDGE $\{\{4, 5\}, 6\}$



The cut defined by $\{\{0, 2\}, 1, 3\}$ still has size 5.

TAKEAWAY The cut associated with S keeps its size as long as we never contract an edge of the cut itself — an edge $\{i, j\}$ with $i \in S$ and $j \notin S$.

KARGER'S ALGORITHM

KARGER'S ALGORITHM

```
KargerMinCut(G):  
  while G has more than 2 vertices:  
    choose an edge {u, v} uniformly at random  
      # among the remaining multi-edges  
    contract {u, v}  
  return the number of edges between the two remaining vertices
```

The two remaining super-vertices define a cut of the original graph; the parallel edges between them are exactly the edges of that cut.

KARGER'S ALGORITHM

If we never choose an edge in some fixed minimum cut, the algorithm returns exactly that minimum cut.

INTUITION A minimum cut is *small*, so a random edge is unlikely to come from it.

SUCCESS PROBABILITY: SETUP

Say the graph G has n vertices and the size of a minimum cut is k . Fix a particular minimum cut $\text{cut}(S)$.

Every vertex has degree at least k — otherwise its star cut would beat the minimum cut.

FACT Summing the degrees, G has at least $nk/2$ edges.

STEP 1: THE FIRST CONTRACTION

The first contraction is bad if the chosen edge lies in $\text{cut}(S)$.

Our cut has k edges out of at least $nk/2$ in total, so the probability the random edge is one of ours is at most

$$\frac{k}{nk/2} = \frac{2}{n}.$$

So the first contraction **misses** our cut with probability at least

$$1 - \frac{2}{n} = \frac{n-2}{n}.$$

STEP 2: AND THE NEXT ONE

Suppose we survived the first contraction. The contracted graph has $n - 1$ vertices and still has min cut size at least k (contraction cannot decrease the mincut value), so it has at least $k(n - 1)/2$ edges.

The probability the second contraction also misses our cut is at least

$$1 - \frac{2k}{k(n - 1)} = 1 - \frac{2}{n - 1} = \frac{n - 3}{n - 1}.$$

STEP 2: AND THE NEXT ONE

So the probability we are still good after two contractions is at least

$$\frac{n-2}{n} \cdot \frac{n-3}{n-1}.$$

STEP 3: IN GENERAL

After the second contraction there are at least $k(n - 2)/2$ edges, so the third contraction misses with probability at least

$1 - \frac{2}{n-2} = \frac{n-4}{n-2}$, giving

$$\frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2}$$

after three contractions.

STEP 3: IN GENERAL

In general, at the i th contraction there are $n - i + 1$ vertices left, and the success probability of that step is at least

$$\frac{n - i - 1}{n - i + 1}.$$

THE TELESCOPING PRODUCT

We do $n - 2$ contractions to reach 2 vertices. The success probability is at least

$$\frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdot \frac{n-5}{n-3} \cdots \frac{3}{5} \cdot \frac{2}{4} \cdot \frac{1}{3} = \frac{2}{n(n-1)}.$$

Every numerator cancels a denominator two steps later — only n and $n - 1$ survive below, and $2 \cdot 1$ above.

LEMMA A single run of Karger's algorithm outputs a fixed minimum cut with probability at least $\frac{2}{n(n-1)}$.

AMPLIFICATION BY REPETITION

Run the algorithm t times independently and return the *minimum* cut value found. We fail only if *every* run fails; by independence,

$$\Pr[\text{all } t \text{ runs fail}] \leq \left(1 - \frac{2}{n(n-1)}\right)^t \leq e^{-2t/(n(n-1))}$$

using our inequality $1 + x \leq e^x$.

AMPLIFICATION BY REPETITION

THEOREM With $t = 3n(n - 1)/2$ repetitions, the failure probability is at most $e^{-3} < 1/10$: we find a minimum cut with probability at least $9/10$.

This is the amplification fact with $p = 2/(n(n - 1))$: $3/p$ repetitions.

IMPLEMENTATION OF ONE RUN

Rather than randomly choosing a remaining edge at each step, it is easier to take an alternative, but equivalent, viewpoint.

We first randomly order the edges; this ordering determines the order in which we contract them. Assign the i th edge in the ordering weight i and run **Kruskal's MST algorithm** (Week 5!).

IMPLEMENTATION OF ONE RUN

The partition of vertices into connected components at any stage represents the super-vertices of the contracted graph.

Before the last edge is added in Kruskal's algorithm there are just two connected components — the edges between these sets are the proposed mincut.

RUNNING TIME

Via Kruskal with union–find, one round of Karger's algorithm runs in time $O(m \log n)$ when there are m edges.

TOTAL With $O(n^2)$ repetitions, the overall running time is $O(n^2 m \log n)$.

Note that during the algorithm we work with a multigraph. We could also *start* from a multigraph — the algorithm works on weighted graphs with non-negative integer weights.

COOL CONSEQUENCE

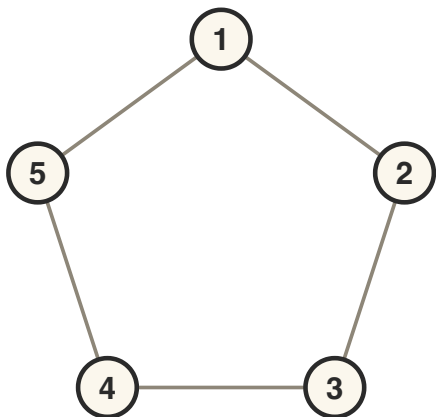
We fixed a minimum cut and argued that the probability *this* cut is preserved by the algorithm is at least $\frac{2}{n(n-1)}$. This holds for *any* minimum cut.

Different minimum cuts being returned are disjoint events, and the probabilities of disjoint events sum to at most 1.

COROLLARY The number of minimum cuts in a graph is at most $\frac{n(n-1)}{2}$.

Remarkably, this bound is exactly tight!

TIGHT EXAMPLE: THE CYCLE GRAPH



In a cycle the size of a minimum cut is 2.

Removing any two edges disconnects the graph, and there are $\binom{n}{2} = \frac{n(n-1)}{2}$ ways to remove two edges.

So the cycle graph C_n has exactly $\frac{n(n-1)}{2}$ minimum cuts — matching the bound.

THE KARGER-STEIN ALGORITHM

HIGH RISK CONTRACTIONS

Look again at the telescoping product, our lower bound on the probability that no contraction ever picks an edge of a fixed minimum cut:

$$\frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdots \frac{2}{4} \cdot \frac{1}{3}.$$

The first factor is $1 - \frac{2}{n}$: the first contraction is almost never bad.

The last factor is $\frac{1}{3}$: the final contraction is bad one time in three.

HIGH RISK CONTRACTIONS

OBSERVATION The risk is concentrated in the last few contractions.

Yet repetition treats every contraction alike: it restarts from scratch $\Theta(n^2)$ times, redoing the safe early part every time.

STOPPING EARLY

What if we stop once t vertices remain, after $n - t$ contractions?

The same product, cut short, bounds the probability that our cut survives:

$$\frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdots \frac{t-1}{t+1} = \frac{t(t-1)}{n(n-1)}.$$

Everything still cancels two steps later; now t and $t - 1$ survive on top. Sanity check: $t = 2$ gives back $\frac{2}{n(n-1)}$.

EARLY CONTRACTIONS ARE SAFE

How far can we contract while keeping the cut with probability at least $1/2$?

Take $t = \lceil n/\sqrt{2} \rceil + 1$. Both t and $t - 1$ are at least $n/\sqrt{2}$, so

$$\frac{t(t-1)}{n(n-1)} \geq \frac{n^2/2}{n(n-1)} \geq \frac{1}{2}.$$

LEMMA Contracting down to $t = \lceil n/\sqrt{2} \rceil + 1$ vertices keeps a fixed minimum cut with probability at least $1/2$.

RECYCLING THE SAFE PART

For $n = 1000$: the first 291 of the 998 contractions keep the cut with probability at least $\frac{1}{2}$. The danger lives in the remaining 707.

Better: contract down to t vertices *once*, then find a minimum cut of the smaller graph — *recursively*.

RECYCLING THE SAFE PART

Each level of recursion shrinks n by a factor of $\sqrt{2}$, so there are about $d = 2 \log_2 n$ levels.

PROBLEM Each level keeps the cut with probability only $\frac{1}{2}$, and the losses compound: $\left(\frac{1}{2}\right)^{2 \log_2 n} = \frac{1}{n^2}$. We are back where we started.

No surprise: contracting all the way down in stages is still just one run of Karger's algorithm.

TWO TRIES AT EVERY LEVEL

A factor of $\frac{1}{2}$ per level is a coin flip per level. Where we risk a coin flip, give ourselves a second one.

At each level, contract to t vertices *twice*, independently, and recurse on both smaller graphs. Return the smaller of the two answers.

Why not three or four tries? We will see that the time budget allows exactly two.

THE KARGER-STEIN ALGORITHM

```
KargerStein(G):  
  n = number of vertices of G  
  if n ≤ 6: return a minimum cut of G by brute force  
  t = ⌈n/√2⌉ + 1  
  G1 = Contract(G, t)  # contract random edges until t vertices remain  
  G2 = Contract(G, t)  # independently of G1  
  return min(KargerStein(G1), KargerStein(G2))
```

The recursion tree is a full binary tree of depth $d \approx 2 \log_2 n$.

[Karger-Stein, 1996]

RUNNING TIME

Store the multigraph as an $n \times n$ table of edge multiplicities.

Contracting an edge merges two rows and two columns: $O(n)$ time.

Choosing a random edge also takes $O(n)$ time: pick an endpoint u with probability proportional to its degree, then a neighbour v with probability proportional to the multiplicity of $\{u, v\}$.

RUNNING TIME

FACT $\text{Contract}(G, t)$ does fewer than n contractions of $O(n)$ time each, so it runs in $O(n^2)$ time.

Each call does two contractions and two recursive calls on graphs with $t \approx n/\sqrt{2}$ vertices:

$$T(n) \leq 2T\left(n/\sqrt{2}\right) + O(n^2).$$

THE RECURSION TREE

Level i has 2^i subproblems, each on about $n/(\sqrt{2})^i = n/2^{i/2}$ vertices.

Each costs $O((n/2^{i/2})^2) = O(n^2/2^i)$, so level i costs $2^i \cdot O(n^2/2^i) = O(n^2)$ in total.

Every level costs the same, and there are $d \approx 2 \log_2 n$ levels.

THEOREM One run of Karger–Stein takes $O(n^2 \log n)$ time.

WHY EXACTLY TWO?

Shrinking to $n/\sqrt{2}$ was forced by wanting survival probability $\frac{1}{2}$.

Two branches of size $n/\sqrt{2}$ cost $2 \cdot \frac{n^2}{2} = n^2$: exactly the cost of the level above.

With three branches the levels would grow geometrically, and the time would be $n^{\log_{\sqrt{2}} 3} \approx n^{3.17}$.

BALANCE Two is the most branching that keeps the cost per level flat.

SUCCESS PROBABILITY

Let p_k be a lower bound on the probability that a call at height k of the recursion tree finds a fixed minimum cut of its input graph.

Leaves (height 0) use brute force: $p_0 = 1$.

To bound p_{k+1} , fix a minimum cut C of the input graph of a call at height $k + 1$, and ask when one of its two branches finds C .

ONE BRANCH

For a branch to find C , two things must happen.

EVENT 1 Contract at height $k + 1$ never chooses an edge of C . By the Lemma this has probability at least $\frac{1}{2}$. Then C is still a minimum cut of the contracted graph.

EVENT 2 C also survives every later round: the recursive call at height k finds C . By the definition of p_k this has probability at least p_k .

SUCCESS PROBABILITY

The recursive call picks fresh random edges, independent of the contraction, so both events happen with probability at least $\frac{1}{2} \cdot p_k$.

So one branch fails with probability at most $1 - \frac{p_k}{2}$.

The call fails only if *both* branches fail, and the two branches are independent:

$$p_{k+1} \geq 1 - \left(1 - \frac{p_k}{2}\right)^2 = p_k - \frac{p_k^2}{4}.$$

HARMONIC, NOT GEOMETRIC

With a single branch we had $p_{k+1} = \frac{p_k}{2}$: halving at every level, down to $2^{-d} = \frac{1}{n^2}$.

With two, $p_{k+1} \geq p_k - \frac{p_k^2}{4}$. Once p_k is small, the loss $\frac{p_k^2}{4}$ is *much* smaller than p_k itself.

CLAIM $p_k \geq \frac{1}{k+1}$ for every $k \geq 0$.

HARMONIC, NOT GEOMETRIC

Induction on k ; the base case is $p_0 = 1$.

The function $x - \frac{x^2}{4}$ is increasing for $x \leq 2$, so $p_k \geq \frac{1}{k+1}$ gives

$$\begin{aligned} p_{k+1} &\geq \frac{1}{k+1} - \frac{1}{4(k+1)^2} \\ &\geq \frac{1}{k+1} - \frac{1}{(k+1)(k+2)} = \frac{1}{k+2}. \end{aligned}$$

The middle step uses $4(k+1) \geq k+2$.

PUTTING IT TOGETHER

The root is at height $d \approx 2 \log_2 n$.

THEOREM One run of Karger–Stein finds a fixed minimum cut with probability at least $\frac{1}{d+1} = \Omega(1/\log n)$, in time $O(n^2 \log n)$.

Amplify as before: $3(d+1) = O(\log n)$ independent runs fail with probability at most $e^{-3} < \frac{1}{10}$.

TOTAL A minimum cut with probability at least $\frac{9}{10}$ in $O(n^2 \log^2 n)$ time.

KARGER VS. KARGER-STEIN

	ONE RUN	SUCCESS	AMPLIFIED
Karger	$O(m \log n)$	$\Omega(1/n^2)$	$O(n^2 m \log n)$
Karger-Stein	$O(n^2 \log n)$	$\Omega(1/\log n)$	$O(n^2 \log^2 n)$

Same contraction step, same telescoping product — the gain comes purely from putting the repetitions where the risk is.

NOTES

Karger's algorithm dates from 1993 and the Karger–Stein refinement from 1996. A different idea, packing spanning trees, gave $O(m \log^3 n)$ time [Karger, 2000].

The current best randomised algorithm runs in time $O(m \log^2 n)$ [Gawrychowski, Mozes, Weimann, 2019], and near-linear time deterministic algorithms are also known.

Next week we meet minimum cuts again, between two designated vertices s and t — and relate them to maximum flows.